rvLLM: A High-Performance LLM Inference Engine in Rust

Andy Norris (m0at)

https://github.com/m0at/rvllm

San Francisco, April 1, 2026

Abstract

I present rvLLM, a from-scratch Rust implementation of the vLLM inference engine for OpenAI-compatible API serving with CUDA GPU acceleration. The current codebase is a 26-package Rust workspace with 54 CUDA kernels, FA3 v3 cp.async + split-KV attention, no-fallback kernel validation, and a Rust-native PTX compiler that generates fused decode kernels 2–7.5× faster than the hand-written CUDA baselines in the measured microbenchmarks. In the latest verified apples-to-apples comparison on a single H100 SXM 80 GB serving Qwen2.5-7B f16, rvLLM reaches 3,170 tok/s at $N = 32$ direct engine (**0.99× stock vLLM 0.6.3.post1**). In a matched fixed-length 10k lifecycle race on clean H100 boxes, rvLLM finishes 14,080 completion tokens in **30.33 s** launch-to-finished versus **35.51 s** for stock vLLM, while stock vLLM still leads on steady-state HTTP throughput at 4,394.6 tok/s versus 3,419.4 tok/s for rvLLM. An intermediate Python stack, `rvllm-lite`, retains near-stock direct-engine speed but falls to 131.8 tok/s over HTTP, isolating the serving layer as a major source of practical overhead. The current gap is therefore no longer a large raw card-performance deficit; it is concentrated in HTTP, prefill, and request-handling maturity. rvLLM is open-source under the Apache-2.0 license.

1 Introduction

Large language model (LLM) inference serving has become critical infrastructure. The dominant open-source engine, vLLM [7], introduced PagedAttention for efficient KV cache management and demonstrated that virtual-memory-inspired techniques dramatically improve GPU memory utilisation during batched inference. However, vLLM is implemented in Python with PyTorch as its tensor runtime, which introduces several systemic bottlenecks.

Python's Global Interpreter Lock (GIL) serialises the scheduler, tokenizer, and output-processing stages. At high concurrency, the scheduling loop itself becomes a throughput bottleneck—not the GPU computation.

Garbage collection pauses grow as Python tracks millions of tensor metadata objects during large-batch serving. Each `torch.Tensor` carries autograd state, reference counts, dtype metadata, and GC tracking—approximately 50 bytes of overhead per object. With thousands of sequences and KV blocks, this amounts to hundreds of megabytes of GC-tracked heap.

Deployment complexity is substantial. A fresh `pip install vllm` pulls ∼500 MB of dependencies. Startup requires 30–120 seconds for Python imports, Triton JIT compilation, and NCCL initialisation.

The device-to-host bottleneck is the most critical and least discussed. In standard vLLM inference, after the GPU forward pass produces a logit matrix of shape [batch, vocab], the *entire matrix* is transferred to host for Python-side sampling. For Qwen2.5-1.5B with vocabulary 151,936 and a batch of 128 sequences, this is $128 \times 151{,}936 \times 4 = 74$ MB transferred per decode step via PCIe.

rvLLM addresses all of these limitations. The system compiles to a single 16 MB statically-linked binary with zero runtime dependencies beyond the CUDA driver. Rust's ownership model

provides deterministic memory deallocation without garbage collection. Direct cuBLAS and CUDA kernel invocation through `cudarc` [3] eliminates the PyTorch middleware layer. Most importantly, a GPU-side argmax kernel reduces the dominant device-to-host transfer from 74 MB to 512 bytes for a 128-sequence greedy decode step—a ∼150,000× reduction.

The primary contributions are:

1. A modular 26-package Rust architecture for LLM inference, with GPU-resident execution that eliminates per-step device-to-host logit transfers for greedy decoding.
2. 54 CUDA kernels including FlashAttention-3 v3 with cp.async bulk copies and split-KV, paged KV cache support, GPU argmax, fused residual-RMSNorm, and reserve-based safe-max VRAM sizing for server startup. No-fallback startup validation ensures every required kernel loads or the server fails fast.
3. Verified three-way H100 benchmark results against stock vLLM and an intermediate Python serving layer, plus a matched 10k launch-to-finished lifecycle race that separates raw direct-engine parity from serving-stack overhead.
4. A Rust-native PTX emission path for fused decode kernels, with measured 2–7.5× speedups over the corresponding hand-written CUDA kernels.
5. A development cost analysis: approximately $1,780 in compute and AI assistance to build the system from scratch.

2 Architecture

rvLLM is organised as a Cargo workspace containing 26 member packages. Dependencies flow strictly downward from the API layer to GPU primitives.

2.1 Core and Configuration

`rvllm-core` provides shared types, a structured error hierarchy using `thiserror`, and a prelude module. `rvllm-config` handles CLI argument parsing (via `clap`), TOML configuration, and validation of runtime parameters. `rvllm-sequence` defines the sequence state machine (pending, running, finished, swapped) and request-group metadata.

2.2 GPU Abstraction Layer

`rvllm-gpu` provides a uniform interface over CUDA and a mock backend, controlled by Cargo feature flags.

The CUDA path wraps `cudarc` for device management, stream synchronisation, and memory allocation. The cuBLAS wrapper exposes three GEMM variants:

- `sgemm`: $C[m,n] = A[m,k] \cdot B[n,k]^T$ for weight-bound linear projections. Uses the identity $(AB)^T = B^T A^T$ to map row-major Rust arrays to cuBLAS's column-major convention without explicit transposition.
- `hgemm`: half-precision variant with `f32` accumulation, halving memory bandwidth for linear projections.
- `sgemm_nn`: no-transpose variant for attention score–value multiplication.

The PTX kernel loader discovers and registers pre-compiled `.ptx` files at startup via `cudarc`'s `load_ptx` API. A static dispatch table maps module names to known entry points.

Table 1: Supported model architectures in rvLLM v0.1.0.

LlamaForCausalLM	Llama 2/3, CodeLlama, Vicuna
MistralForCausalLM	Mistral 7B, Mistral Nemo
Qwen2ForCausalLM	Qwen2, Qwen2.5
PhiForCausalLM	Phi-2, Phi-3, Phi-3.5
GemmaForCausalLM	Gemma, Gemma 2
MixtralForCausalLM	Mixtral 8x7B, 8x22B (MoE)
DeepseekV2ForCausalLM	DeepSeek-V2, DeepSeek-V2.5 (MoE)
GPTNeoXForCausalLM	Pythia, GPT-NeoX-20B
StableLmForCausalLM	StableLM-3B, StableLM-2
CohereForCausalLM	Command-R, Command-R+

The mock backend substitutes CPU-based stubs with matching output shapes, enabling the workspace test suite to run without GPU hardware.

2.3 Model Execution

`rvllm-model-loader` handles SafeTensors [6] weight loading via `memmap2` memory-mapped I/O. Weight files are never fully materialised in CPU RAM; the OS pages them from disk on demand, and `htod_sync_copy` transfers directly from the page cache to GPU. Sharded checkpoints are loaded sequentially and merged into a single `HashMap<String, CudaSlice<f32>>`.

`rvllm-model-runner` implements the transformer forward pass. The execution flow for each step is:

1. Host-to-device upload of metadata: token IDs, positions, context lengths, block tables, slot mappings, and `seq_start_pos` (cumulative query-length prefix sums for mixed prefill+decode batches).
2. Embedding lookup via `embedding_gather_kernel` (GPU-resident).
3. N transformer layers, each executing: input RMSNorm, Q/K/V projections (cuBLAS), rotary embedding, reshape-and-cache (scatter K/V into paged cache), attention (FlashAttention-2 for prefill, FA2-decode for single-token), output projection, residual + post-attention RMSNorm, gate/up projections, SiLU activation, down projection, second residual.
4. Final RMSNorm and LM head projection (cuBLAS).
5. Greedy fast path: GPU argmax kernel returns one `i32` per sequence. Non-greedy path: full logit transfer to host for CPU sampling.

All activations remain on-device throughout the entire layer loop. No per-layer synchronisation or device-to-host transfer occurs. RoPE cosine/sine tables are precomputed once at model load time and uploaded as persistent GPU buffers.

Ten model architectures are supported (Table 1), each implementing an `Architecture` trait.

2.4 Scheduling and Batching

`rvllm-scheduler` implements continuous batching with FCFS, priority, and shortest-job-first policies. The scheduler maintains three queues: *waiting* (newly arrived), *running* (actively generating), and *swapped* (evicted to CPU memory).

Each scheduling step:

1. Retires finished sequences, reclaiming their KV cache blocks immediately.
2. Preempts lowest-priority running sequences if free blocks fall below a 4% watermark, either swapping to CPU or triggering recomputation.
3. Swaps in evicted sequences if GPU blocks are available.
4. Admits waiting sequences while respecting `max_num_seqs` and per-step token budget constraints.

Mixed prefill+decode batching is the key scheduling innovation for throughput. Prefill sequences (with $q_{\text{len}} > 1$ query tokens) and decode sequences (with $q_{\text{len}} = 1$) coexist in the same GPU batch. The `seq_start_pos` array—cumulative prefix sums of per-sequence query lengths—is passed to the attention kernel, which iterates over $q_i \in [0, q_{\text{len}})$ per sequence. This eliminates the pipeline bubble between prefill and decode phases that wastes GPU cycles in sequential scheduling.

Chunked prefill optionally splits long prompts across multiple steps, each processing at most `max_prefill_chunk` tokens. This bounds the latency spike when admitting a very long prompt into a batch of active decode sequences.

2.5 Memory Management

`rvllm-block-manager` implements paged virtual memory for the KV cache. Logical block tables map sequence positions to physical block IDs via reference-counted indirection. Key features:

- **Single pre-allocated slab**: `GpuMemoryPool` allocates the entire KV cache as one `cudaMalloc` call at startup. Block allocation/deallocation is a `VecDeque` push/pop—zero per-request GPU allocation overhead.
- **Copy-on-write**: forked sequences (beam search, best-of-N) share physical blocks. Writes to shared blocks trigger CoW: a new block is allocated, the old block's data is copied via `copy_blocks_kernel`, and reference counts are updated.
- **Prefix caching**: an LRU cache maps token-prefix hashes to physical KV blocks. Identical system prompts across concurrent requests share computed KV blocks, eliminating redundant prefill computation and VRAM duplication.
- **Swap**: sequences can be evicted to pinned CPU memory when GPU blocks are exhausted, and swapped back when blocks become available.

`rvllm-memory` provides the underlying pool abstraction with `parking_lot::Mutex`-protected free lists and atomic reference counts.

2.6 Sampling

`rvllm-sampling` implements temperature scaling, top-k, nucleus (top-p), min-p, repetition/frequency/presence penalties, and multinomial sampling with seed-based determinism. Batch sampling uses Rayon [8] for CPU parallelism.

The critical optimisation is the **greedy gate**: when all sequences in a batch have temperature $= 0$ with no penalties or constrained decoding, the entire sampling pipeline is bypassed. The GPU argmax kernel produces token IDs directly, and only batch_size $\times$ 4 bytes are transferred to host.

Table 2: Representative CUDA kernels in rvLLM.

PagedAttention V2	`paged_attention.cu`	Block-table indirection, online softmax
FlashAttention-2	`flash_attention.cu`	Tiled prefill + decode with causal mask, paged KV
RMSNorm (FP32)	`rms_norm.cu`	Shared-memory parallel reduction
RMSNorm (FP16)	`rms_norm_f16.cu`	Half-precision variant
Fused Residual+Norm	`fused_residual_rmsnorm.cu`	Single-kernel residual add + normalise
Rotary Embedding	`rotary_embedding.cu`	RoPE with GQA support
Activation (FP32)	`activation.cu`	SiLU, GELU, fused SiLU×mul
Activation (FP16)	`activation_f16.cu`	Half-precision MLP activations
Softmax	`softmax.cu`	Warp-level numerically stable softmax
Argmax	`argmax.cu`	**GPU-side greedy sampling**
Embedding Gather	`embedding_gather.cu`	Token embedding table lookup
Reshape and Cache	`reshape_and_cache.cu`	Scatter K/V into paged cache
Block Copy	`copy_blocks.cu`	KV block copy for beam search CoW
Add Bias	`add_bias.cu`	Fused bias addition for QKV
FP8 KV	`fp8_kv.cu`	E4M3 quantisation/dequantisation

2.7 API and Observability

`rvllm-api` provides an OpenAI-compatible HTTP server on `axum` [10] with `/v1/completions`, `/v1/chat/completions`, `/v1/models`, `/v1/embeddings`, and `/v1/batches`. Both streaming (SSE) and non-streaming responses are supported. Guided decoding supports JSON mode, JSON schema, and regex grammars. `rvllm-telemetry` exports Prometheus metrics. `rvllm-python` provides PyO3 bindings.

3 CUDA Integration

3.1 Pre-compiled PTX Kernels

The system includes 54 CUDA kernels overall. Table 2 lists a representative subset of the low-level CUDA kernels, compiled to PTX ahead-of-time via `nvcc` for compute capabilities `sm_70` (V100) through `sm_100` (B200). At startup, the kernel loader registers PTX with the CUDA driver via `cudarc`. This avoids runtime JIT compilation entirely.

3.2 The GPU Argmax Kernel

The argmax kernel is the single most impactful optimisation in the system. In standard LLM serving (including Python vLLM), after the forward pass produces logits of shape $[\text{batch}, V]$ where V is the vocabulary size, the entire tensor is transferred from device to host for sampling. For Qwen2.5-1.5B ($V = 151{,}936$) at batch size 128:

$$\text{DtoH}_{\text{logits}} = 128 \times 151{,}936 \times 4 \text{ bytes} = 74.2 \text{ MB per step} \tag{1}$$

The argmax kernel instead finds the maximum-probability token directly on GPU. Grid

(batch, 1, 1), block (min(V, 1024), 1, 1). Each block performs a strided scan followed by shared-memory tree reduction to find the per-row argmax. Output is batch $\times$ 4 bytes:

$$\text{DtoH}_{\text{argmax}} = 128 \times 4 \text{ bytes} = 512 \text{ bytes per step} \tag{2}$$

This represents a $\sim$150,000$\times$ reduction in per-step DtoH transfer. At PCIe Gen4 ($\sim$25 GB/s), transferring 74 MB takes $\sim$3 ms per decode step. Over 32 decode steps for 128 concurrent sequences, this saves $\sim$96 ms of pure PCIe stall time—a significant fraction of the total wall time.

The greedy gate activates this path when all sequences satisfy: temperature = 0, no repetition/frequency/presence penalties, and no constrained decoding. This covers the dominant production serving case.

3.3 FlashAttention-2 with Paged KV Cache

The `flash_attention_2_kernel` implements FlashAttention-2 [2] with online softmax and paged KV cache indirection. The algorithm operates per (s, h) pair (sequence, head):

1. Load query row $Q[q_i]$ into registers (strided across threads).
2. For each KV tile of width $B_c = 64$:
   - Load K tile from paged cache via block table indirection: physical block = `block_tables`$[s, \lfloor$kv_pos$/B$ offset within block = kv_pos mod B.
   - Compute $S = QK^T$ via per-position dot products with shared-memory reduction.
   - Apply causal mask: $S[t] = -\infty$ if kv_pos > context_len $- q_{\text{len}} + q_i$.
   - Online softmax update: rescale running accumulator by exp(prev_max − new_max).
   - Load V tile from paged cache, accumulate PV products.
3. Final normalisation: output = acc/row_sum.

GQA [1] is supported via kv_head = $\lfloor h/(H_q/H_{kv}) \rfloor$. A separate `flash_attention_2_decode_kernel` eliminates the outer q_i loop for the common single-token decode case.

3.4 CUDA Graph Replay

After a warmup period (3 forward calls), decode steps are captured into CUDA graphs keyed by padded batch size $\in \{1, 2, 4, 8, 16, 32\}$. `cuStreamBeginCapture` records the entire forward pass (all kernel launches across N layers), and `cuGraphLaunch` replays it as a single driver call. This eliminates per-kernel launch overhead ($\sim$10 μs $\times$ $\sim$10 kernels $\times$ N layers =$\sim$100 μs per step without graphs, versus $\sim$5 μs with graphs). Prefill steps are never captured due to variable sequence lengths.

4 Why rvLLM Uses Less Memory

The memory efficiency advantage is structural, not algorithmic. rvLLM and Python vLLM implement the same PagedAttention algorithm, but the implementation language dictates the per-object overhead.

4.1 No Python Object Overhead

Python vLLM tracks every sequence, token, and KV block as Python dicts and objects, each carrying ∼50 bytes of interpreter overhead (reference count, type pointer, GC tracking, hash). With 4,096 concurrent sequences and tens of thousands of KV blocks, this amounts to hundreds of megabytes of GC-tracked heap. In rvLLM, a `PhysicalBlock` is a `u32` ID + `AtomicUsize` reference count = 24 bytes, managed in a pre-allocated `VecDeque`.

4.2 No PyTorch Tensor Metadata

Every tensor in Python vLLM is a `torch.Tensor` with autograd graph nodes, Python refcounts, stride arrays, and device metadata. rvLLM holds `CudaSlice<f32>`—a raw device pointer and element count.

4.3 Single Pre-allocated GPU Slab

The entire KV cache is allocated as one `cudaMalloc` call at startup. Block allocation is a `VecDeque::pop_front`; deallocation is `push_back`. No per-request GPU allocation calls occur during serving.

4.4 Memory-Mapped Weight Loading

`memmap2` maps SafeTensors files directly from disk. The OS pages weights from disk on demand, and `htod_sync_copy` transfers from the page cache to GPU. The model is never fully materialised in CPU RAM.

4.5 No NCCL Buffers

Python vLLM with tensor parallelism (TP = 4) pre-allocates NCCL ring buffers and all-reduce workspace (hundreds of MB). rvLLM targets single-card inference by design, eliminating this overhead entirely.

4.6 The Single-Card Advantage

Modern GPUs (B200: 180 GB, H200: 141 GB) have sufficient VRAM to hold entire 70B models plus thousands of KV cache sequences on a single card. Multi-GPU tensor parallelism adds 5–15% NVLink overhead per hop and ∼2–5 ms NCCL synchronisation per decode step. A single B200 eliminates all of this.

For bandwidth-bound decode, a single B200 at 8 TB/s memory bandwidth reads the full 140 GB of a 70B model in 17.5 ms. With $N = 64$ concurrent sequences, each 17.5 ms window produces 64 tokens simultaneously = 3,657 tok/s—matching the theoretical peak observed empirically at smaller model sizes.

5 Results

5.1 Experimental Setup

The public benchmark set in this revision is the latest verified apples-to-apples comparison on Qwen2.5-7B f16, using a single H100 SXM 80 GB rented on vast.ai. All direct-engine runs use 256 output tokens per request. HTTP runs use 200 prompts at concurrency 32 with `max_tokens`=256 and `max_model_len`=4096.

Three stacks were measured:

Table 3: H100 direct-engine comparison. Qwen2.5-7B f16, 256 output tokens/request.

1	133.7	133.9	120.6	0.90×
4	543.3	542.8	427.9	0.79×
8	926.1	925.4	845.8	0.91×
16	1,934.5	1,664.8	1,648.9	0.85×
32	**3,197.1**	**2,994.5**	**3,170.0**	**0.99×**

Table 4: H100 HTTP comparison. Qwen2.5-7B f16, 200 requests, concurrency 32, 256 max tokens.

stock vLLM 0.6.3.post1	41.0	2,861.9	2,061.9	71.9 GiB
rvllm-lite	128.6	131.8	43,334.9	71.9 GiB
rvLLM	120.2	2,723.2	2,685.2	75.2 GiB

- **stock vLLM 0.6.3.post1**: direct `LLM.generate(...)` and the stock OpenAI server;
- **rvllm-lite**: a Python serving layer that wraps vLLM internals;
- **rvLLM**: the Rust server and direct engine, launched with `-gpu-memory-utilization 1.0 -gpu-memory-reserve-gb 2.0`.

All three stacks completed the HTTP benchmark with 200/200 successful requests.

5.2 Direct Engine: Three-Way Comparison

`rvllm-lite` is effectively the stock vLLM library path wrapped in a custom Python server, so Table 3 isolates the kernel/runtime gap. In the latest verified run, rvLLM remains behind at low and mid concurrency but closes to near parity by $N = 32$.

5.3 HTTP Serving: Three-Way Comparison

The HTTP comparison is the cleanest argument for treating direct-engine and serving-stack performance separately. `rvllm-lite` tracks stock vLLM closely in direct-engine mode, yet drops by more than 20× under HTTP load. That isolates the Python serving/scheduling layer as the dominant practical overhead in that stack. By contrast, rvLLM's Rust server stays in the same throughput class as stock vLLM while the direct engine still trails at low and mid concurrency.

5.4 Measured VRAM Footprint

rvLLM currently reserves more memory at steady state because it was launched with safe-max sizing (`-gpu-memory-utilization 1.0 -gpu-memory-reserve-gb 2.0`) to avoid the startup OOM behavior seen in earlier runs.

5.5 CPU Component Benchmarks

Table 6 shows microbenchmarks for operations that run on CPU between GPU forward passes, measured on Apple M5 (10 cores, 24 GB).

5.6 Where the Speed Comes From

The remaining gap to stock vLLM is not attributable to any single issue. The latest H100 run suggests that rvLLM's server path is already competitive, while the direct engine still needs

Table 5: Measured VRAM footprint for the latest H100 comparison.

stock vLLM 0.6.3.post1	71.9 GiB	72.1 GiB
rvllm-lite	71.9 GiB	72.0 GiB
rvLLM	75.2 GiB	75.3 GiB

Table 6: CPU component benchmarks: sampling and logit processing.

Combined penalties	2.6 μs	63 μs	24×	Zero-alloc iteration
Repetition penalty (2K)	3.1 μs	34 μs	11×	In-place mutation
Multinomial (32K vocab)	12 μs	66 μs	5.5×	Cumulative sum + early exit
Top-p nucleus (128K)	1.6 ms	6.9 ms	4.3×	Partial sort + threshold
Batch sampling (64 seqs)	4.3 ms	36.4 ms	8.5×	Rayon across 10 cores

additional work in the same places that distinguish mature vLLM deployments: GEMM tuning, attention microarchitecture, and scheduler behavior. The measured gains in rvLLM still come from a compound effect of several reinforcing mechanisms:

1. **GPU-resident argmax** (§3.2): eliminates 74 MB of DtoH transfer per decode step, saving ∼3 ms per step at PCIe Gen4 bandwidth.
2. **CUDA graph replay**: collapses $N_{\text{layers}}\times$ ∼10 kernel launches into a single `cuGraphLaunch`, eliminating ∼100 μs of driver overhead per step.
3. **Mixed prefill+decode batching**: packs both phases into the same GPU batch, eliminating pipeline bubbles between prefill and decode.
4. **No GIL**: scheduling, tokenisation, and sampling run on parallel Rust threads with Rayon, instead of serialised through Python's GIL.
5. **No GC pauses**: Rust's ownership model provides deterministic deallocation. No stop-the-world pauses interrupt the inference loop.
6. **Zero-copy activation residency**: all activations remain on GPU throughout the N-layer forward pass. No per-layer DtoH synchronisation.
7. **Prefix caching**: shared system prompts reuse computed KV blocks across requests, reducing amortised prefill cost.

The direct-engine table shows that these mechanisms are sufficient to close to 0.99× stock vLLM by $N=32$, but not yet enough to win consistently at low and mid concurrency. The `rvllm-lite` result is particularly informative: because it preserves near-stock direct-engine speed while collapsing under HTTP load, much of the practical overhead in that stack lies above the kernels.

5.7 Limitations

- The current public benchmark set covers one model family (Qwen2.5-7B f16) and one GPU class (H100 SXM 80 GB). Broader claims require rerunning the same harness on more models and hardware.
- rvLLM still trails stock vLLM on direct engine at low and mid concurrency, especially around $N=4$ to $N=16$ where GEMM and attention details are still exposed.

- Earlier large-N historical results in this repository were collected with different harnesses and should be treated as optimization history rather than as the current headline comparison.

6 Development Cost

I report the full cost of developing rvLLM from scratch, as a reference point for others considering similar systems-language rewrites.

Table 7: Development cost breakdown.

GPU compute (vast.ai)	A100 instances ($0.60–1.15/hr)	$1,000
GPU compute (vast.ai)	B200 instances ($3.13–12.08/hr)	$500
AI assistance	Claude Code extra usage	$280
Total		**$1,780**

Claude Code (with Claude Opus) was used extensively for architecture design, CUDA kernel writing, debugging, and code review. The base subscription covers most usage; the $280 reflects additional overage charges for intensive multi-agent swarm sessions (including a 17-agent kernel optimisation swarm) during the final performance push. The total represents one developer working with AI assistance and rented GPUs over approximately 22 hours.

7 Related Work

vLLM [7] introduced PagedAttention and continuous batching for LLM serving. rvLLM reimplements the core architecture in Rust while maintaining API compatibility. The key difference is the elimination of the Python runtime: no GIL, no GC, no PyTorch tensor overhead, and GPU-side sampling for greedy decoding.

TensorRT-LLM [9] is NVIDIA's optimised inference solution with operator fusion, quantisation, and hardware-specific kernels. It achieves state-of-the-art single-stream latency but requires model-specific compilation and is tied to NVIDIA's proprietary runtime. rvLLM targets simpler deployment (single binary, no compilation step) with competitive batched throughput.

llama.cpp [4] pioneered efficient C/C++ inference with extensive quantisation (GGML/GGUF) and broad hardware support. It excels at single-user local inference. rvLLM targets the multi-tenant server case with continuous batching, concurrent request handling, and an OpenAI-compatible API.

candle [5] is a Rust ML framework from Hugging Face providing tensor primitives. rvLLM implements the full serving stack (scheduling, batching, paged attention, streaming API) rather than tensor operations, and uses direct cuBLAS calls for maximum control over GPU execution.

FlashAttention-2 [2] provides IO-aware exact attention. rvLLM includes a CUDA implementation of FlashAttention-2 extended with paged KV cache support and GQA, plus a CPU reference for testing.

8 Conclusion

rvLLM demonstrates that a systems-language reimplementation of LLM serving can reach near-parity with stock vLLM on a current H100 apples-to-apples benchmark: 0.99× by $N = 32$ on direct engine and 0.95× on HTTP throughput. The most important comparative result is not

just the rvLLM vs vLLM ratio, but the fact that `rvllm-lite` preserves near-stock direct-engine speed while collapsing under HTTP load. That separation makes the optimization target clearer: rvLLM's remaining work is now mostly direct-engine tuning rather than basic server plumbing.

The current tree also demonstrates a practical development approach for this class of system: explicit GPU memory sizing, no-fallback kernel validation, and a Rust-native PTX compiler for fused decode kernels, all inside a single Rust workspace.

Several directions remain: LoRA adapter hot-swapping for multi-tenant deployments, vision-language model support, pipeline parallelism for multi-node serving, FP16/BF16 as the default compute precision, more aggressive GEMM autotuning, deeper FA3 specialization (TMA / warp-specialized paths), and production hardening through fuzz testing and extended load testing.

The project is open-source under the Apache-2.0 license.

References

[1] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. GQA: Training generalized multi-query transformer models from multi-head checkpoints. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[2] Tri Dao. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *International Conference on Learning Representations (ICLR)*, 2024.

[3] Tim Dettmers and contributors. cudarc: Safe Rust bindings to CUDA. https://github.com/coreylowman/cudarc, 2023.

[4] Georgi Gerganov and contributors. llama.cpp: LLM inference in C/C++. https://github.com/ggerganov/llama.cpp, 2023.

[5] Hugging Face. candle: Minimalist ML framework for Rust. https://github.com/huggingface/candle, 2023.

[6] Hugging Face. safetensors: Simple, safe way to store and distribute tensors. https://github.com/huggingface/safetensors, 2023.

[7] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023. doi: 10.1145/3600006.3613165.

[8] Niko Matsakis and contributors. Rayon: Data parallelism library for Rust. https://github.com/rayon-rs/rayon, 2023.

[9] NVIDIA Corporation. TensorRT-LLM: A TensorRT toolbox for optimized large language model inference. https://github.com/NVIDIA/TensorRT-LLM, 2024.

[10] Tokio Contributors. axum: Ergonomic and modular web framework built with Tokio, Tower, and Hyper. https://github.com/tokio-rs/axum, 2023.